

INSuRE Spring 2023 Project Proposal

Malware Similarity

Sponsor: MITRE

Frank Posluszny
MITRE
Technical Director
malwarefrank@mitre.org

Brian Ricks
UTD
Faculty
bwr031000@utdallas.edu

Neelima Reddy Kapa
UTD
NXK220027@utdallas.edu

Venu Gopal Reddy Yendreddy
UTD
VXY210021@utdallas.edu

Navya Teja Gajula
UTD
NXG220002@utdallas.edu

December 10th, 2023

CONTENTS

S.No		Page No
1	Project Summary	3
1.1	Top Level Project Description	3
1.2	Keywords	3
1.3	Terminology	3
1.4	Team Member Responsibilities	4
1.5	Estimated Project Budget	4
1.6	Summary of Key Deliverables	5
2.	Executive Summary	6
3.	Motivation	7
4.	Literature Review	8
4.1	Malware Similarity Analysis: An adversary perspective	8
4.2	Software Forensics – Can we track code to its authors?.	9
4.3	Automatic Analysis Of Malware Behaviour Using Machine Learning	10
5.	Specific Aims	11
6.	Research Plan/Approach	12
7.	Anticipated Issues and Challenges	14
8.	Broader Impact	15
9.	Final Progress	16
9.1	Heuristics Phase 1	17
9.2	Heuristics Phase 2	19
9.3	Heuristics Phase 3	19
9.4	SSDeep	20
9.5	Functionality	21
9.6	Findings	22
9.7	Summary	23
9.8	Challenges	23
9.9	Future Work	24
10	Investigator Biographies	25
11	References	26

1. Project Summary

1.1 Top Level Project Description

Our project, guided by MITRE's Technical Director Frank Posluszny, delves into the fascinating realm of cybersecurity. We're on a mission to connect professionals working on similar malware projects across the country. By identifying the unique fingerprints left by the Malware Development Life Cycle (MDLC), we aim to foster collaboration and reduce duplication of effort. Through this project, we envision a future where cybersecurity experts come together seamlessly, enhancing our collective ability to safeguard digital landscapes. This initiative not only bolsters our understanding of cyber threats but also propels us towards a more secure and interconnected digital world.

1.2 Keywords

Malware Similarity, Malware Analysis, Malware Development Life Cycle (MDLC), Hashing, CTPH(Context Triggered Piecewise Hashing, Fuzzy Hashing, SSDEEP.

1.3 Terminology

Malware: A piece of software that performs malicious actions.

Malware Analysis: The process of understanding the behaviour, original, and impact of a malware sample.

Static Analysis: The analysis of programs that involves examining the code without actually executing it.

Dynamic Analysis: The analysis of programs by monitoring the program when it is executed.

MDLC: A new malware analysis methodology developed at MITRE that aims to explore the human element of malware development and how it can be exploited to both gain information about malware samples and identify malware developers.

Hashing: Hashing is a technique used in database management systems to search for the location of data without making use of an index structure. It makes it easy to determine whether or not two files in a computer system are the same. The retrieval and processing of data can be done very quickly with hash tables.

CTPH: A CTPH is a rolling hash and involves multiple traditional cryptographic hashes for one or more fixed-size segments in a file.

Fuzzy Hashing: Fuzzy hashing is a type of compression function for calculating the similarity between digital files. It attempts to automate the process of grouping similar malware.

1.4 Team Member Responsibilities

Venu Gopal Reddy : Will serve as a team leader of the project and will be in charge of scheduling group meetings and reaching deadlines and will be in touch with our technical director of this project for further discussions . He has knowledge of technical aspects of the project and had an experience in c , c++ , python ,java which will be used in creating own scripts to develop this project .

Neelima Reddy Kapa: She is the assistant team lead for the project and in charge of organizing team internal meetings and collaborations on teams . She is well versed with machine learning libraries and python programming.

Navya Teja Gajula : She will be the researcher based on the project needs and she has experience in java , python . She is proficient in data analysis and will focus in incorporating the knowledge for this project.

1.5 Estimated Project Budget

In terms of time budget, this project is estimated to consume around 533 hours, accounting for 8 hours per week per researcher and 1 hour per week of the technical director's time. Currently, there are no monetary expenses required for the project, as we are focusing on conducting more research to better understand the various avenues of our proposed statements. We do not think that there will be any software in the implementation phase that will cost money.

1.6 Summary of Key Deliverables

Key Deliverables	Proposed Due Date
Project Proposal	08/08/2023
Literature Review	08/15/2023
Linux and Windows VM setup	10/20/2023
Static Analysis	10/27/2023
Midterm Presentation	10/13/2023
Midterm Report	10/13/2023
Traditional Hashing	11/17/2023
Final Presentation	08/12/2023
Final Report	10/12/2023

2. Executive Summary

Our project's main goal was to strengthen our cybersecurity defences by investigating and implementing the Malware Development Life Cycle (MDLC). Our strategy was guided by the MDLC, a thorough framework that documents the stages from the creation of malicious software to its deployment. By breaking down the life cycle, we were able to position our defences to be resilient and adaptive and to obtain an in-depth knowledge of the dynamic threat landscape.

A key component of our work was static analysis, which offered a non-execution-based analysis of the structure and code of malware. With the help of this technique, we were able to identify important characteristics like code patterns and artifacts, which provided insightful information about the possible threats. We used advanced techniques such as hashing and fuzzy hashing to enhance static analysis. Hashing made it easier to create distinct file fingerprints, which made it possible to quickly identify known malicious entities. Meanwhile, fuzzy hashing improved our analytical abilities by enabling us to identify differences in known malware, even when it has undergone minor changes.

Significant results were obtained by combining static analysis, advanced hashing techniques, and MDLC knowledge. Our cybersecurity posture has been strengthened by this proactive approach, which also provides actionable intelligence to effectively mitigate risks. To sum up, our project improved our understanding of the MDLC and provided us with useful tools for continuing malware analysis, ensuring a strong defense against new online threats.

3. Motivation

The motivation behind starting this semester-long expedition into malware analysis was a constant commitment. The dynamic character of cyber threats posed a significant obstacle, sparking a shared ambition to not only address occurrences but also proactively detect and eliminate possible risks. We became interested in the nuances of MDLC because we realized that the threat landscape is constantly changing and that it can be used as a strategic compass to navigate this ever-changing environment. We were driven by a common conviction that, by comprehending the malware lifecycle, we could equip ourselves with the information and resources required to properly protect our digital assets.

Furthermore, our project was motivated by the possibility of making a contribution to the larger field of cybersecurity, where every analysis and discovery strengthens the group's defense against cyberattacks. Our commitment was sparked by the realization that our efforts had practical applications. This drive ensured that every step in the analysis process and commitment into chances for learning and development.

Our inspiration came, essentially, from the realization that the digital world necessitates ongoing creativity, ingenuity, and teamwork. We were able to overcome the challenges of MDLC, static analysis, and sophisticated hashing techniques because of our common goal, which transformed what could have been a semester-long project into a journey of empowerment, education, and support for the ongoing fight against cyber threats.

4. Literature Review

4.1 Malware Similarity Analysis: An adversary perspective

Article type: Research Paper

Authors: Frank Posluszny

The research paper by Frank Posluszny, titled "Software Forensics," introduces the innovative concept of the Malware Development Life Cycle (MDLC). This methodology, inspired by the Software Development Life Cycle, consists of eight distinct phases: Requirements, Design, Code, Build, Configure, Package, Collect, and Analyze. The MDLC is designed to uncover the traces left by malware creators and operators, providing a comprehensive framework for understanding the life cycle of malicious software.

The first six phases of the MDLC are emphasized as being under the control of adversaries, illustrating the intentional and strategic decisions made during the development and deployment of malware. These phases include defining requirements, designing the malicious software, writing code, building the executable, configuring the malware, and packaging it for deployment. The remaining two phases, Collection and Analysis, are not directly controlled by adversaries but significantly impact the behaviour and effectiveness of the malware.

The article suggests that practitioners in malware analysis can leverage MDLC to identify similarities among different instances of malware. This involves examining patterns in coding style, authorship, and the overall behaviour of the malware, allowing analysts to draw connections and gain insights into the goals and tactics of adversaries. The MDLC is positioned as a valuable tool when integrated with existing frameworks, providing a structured approach to understanding and mitigating the threats posed by malicious software.

Furthermore, the paper highlights the synergy between MDLC and the Diamond Model of Intrusion, emphasizing how this combination can contribute to the development of more robust defense and mitigation strategies. By revealing connections between different malicious programs, analysts can enhance their understanding of adversary behaviour, ultimately making it more challenging for adversaries to achieve their objectives.

In conclusion, the article calls for further research to expand upon the concepts introduced in the MDLC framework. The goal is to identify measurable artifacts

associated with each stage of the MDLC, providing a more concrete foundation for practitioners in the field of malware analysis. Frank Posluszny's research paper is acknowledged as a significant contribution to the realm of digital forensics and software analysis, providing valuable insights and methodologies for researchers and practitioners in cybersecurity.

4.2 Software Forensics

Article Type : Research Paper

Authors : Eugene H. Spafford and Stephen A. WeebeT

Anonymity is a key aspect that makes both computer crime and computer vandalism attractive activities. Whether the attack method involves viruses, worms, logic bombs, or unauthorized account access, it's generally very challenging to trace these actions back to specific individuals. An example is a hacker mentioned in a well-known case who was not worried about being caught even after detection, as he trusted that no one could trace his actions due to multiple network hops. Authors of viruses distribute them without fear of being identified.

To deter attackers, it's essential to reduce anonymity in computer use through measures like comprehensive session logging and improved network protocols with identification information. But there are limitations to the effectiveness of these methods, and no approach is entirely fool proof. Handwriting analysis involves identifying specific features in writing, like the way letters are formed or the size of characters. Writer-specific features, which show small variations within an individual's writing and significant variations across different authors, are critical in identifying handwriting. It requires multiple samples to identify these features accurately, as a person's handwriting changes over time and under different conditions. Similarly, identifying authorship in computer code is possible by matching identifying characteristics. While code lacks physical features like handwriting, it offers room for variation and personalization, even when following detailed specifications. Programmers often have unique coding styles, and many texts recommend coding style elements. Statistical inference techniques, which have been used to verify authorship in prose, can also be applied to software, reducing subjectivity in the analysis. Identifying the author of suspect code requires selecting an appropriate code sample and identifying relevant features for comparison. This may be challenging if the programmer attempted to conceal their authorship or if suitable sample code is unavailable. Analyzing code characteristics may lead to suspect identification and insights into attack methods and vulnerabilities. With sufficient background research and a large enough code sample, statistical methods applied to prose authorship determination may also be applicable to code. These methods, while not

infallible, can potentially complement the analysis of stylistic features to provide clues about the authorship of code.

4.3 Automated Analysis of Malware Behavior Using Machine Learning

Article Type: Research Article

Authors: Rieck, Konrad; Trinius, Philipp; Willems, Carsten; Holz, Thorsten

Malicious software, or malware, poses a substantial threat to computer system security, with the sheer volume and diversity of variants rendering traditional security defenses ineffective. This article introduces a framework for the automated analysis of malware behavior using machine learning. In response to malware's evasion techniques like obfuscation and polymorphism, the focus is on dynamic analysis during runtime, providing insights for effective defense. The framework facilitates automatic identification and categorization (clustering) of new malware types based on behavior, along with the classification of unknown malware into these newly identified categories. The article advocates for an incremental approach to behavior-based analysis, capable of efficiently processing the behavior of thousands of malware binaries daily. This incremental analysis reduces runtime overhead compared to current methods, ensuring accurate discovery and discrimination of new malware variants

5. Specific Aims

In pursuit of these specific aims, our report aims to employ advanced techniques in cybersecurity to investigate and map the connections between diverse malware strains. This involves not only identifying shared code snippets or common functionalities but also exploring potential collaborations or affiliations among threat actors. Additionally, we intend to conduct a thorough examination of the evolution of malware families and their variants over time. By tracing their development trajectories, we aim to uncover key milestones, modification patterns, and adaptations, providing a nuanced understanding of how these malicious entities morph and propagate.

On the behavioral front, our report seeks to conduct a granular analysis of the actions and interactions exhibited by various malware types. This involves scrutinizing their modus operandi, such as propagation methods, persistence mechanisms, and evasion tactics. Through this behavioral profiling, we aim to discern patterns that can be utilized for predictive analysis. This proactive approach will empower cybersecurity professionals to anticipate potential threats, identify emerging trends, and fortify defenses based on a deeper comprehension of the recurrent behaviors exhibited by malware.

In essence, our comprehensive investigation and behavioral profiling endeavor to contribute not only to the theoretical understanding of malware relationships and behaviors but also to the practical development of more effective and adaptive cybersecurity strategies. This report aims to serve as a valuable resource for practitioners, researchers, and organizations seeking to stay ahead of the ever-evolving landscape of cybersecurity threats.

6. Research Plan/Approach

The project will unfold in three distinct phases. The initial phase, starting after the completion of the project proposal, will involve each team member conducting a comprehensive literature review to explore potential approaches addressing the problem outlined in Section 5. Collaborative efforts with our mentor will document findings and challenges, discussed in regular weekly meetings. The objective for this phase is to enhance our understanding of the problem domain and determine the solutions for implementation.

The second phase centres around prototyping the selected approach. Specifics remain uncertain until after phase one, but it will likely involve a design phase outlining solution steps and an implementation phase putting the plan into action. The approach may align with methods in Section 5 or a novel discovery from the literature review. By the end, our goal is a functional prototype or case study representing our chosen approach for effective presentation.

The third and final phase involves a comprehensive review of the initial approach with necessary revisions. Anticipated to be relatively brief, this phase addresses any issues with the chosen method and identifies potential challenges, providing valuable insights for future research in malware similarity detection.

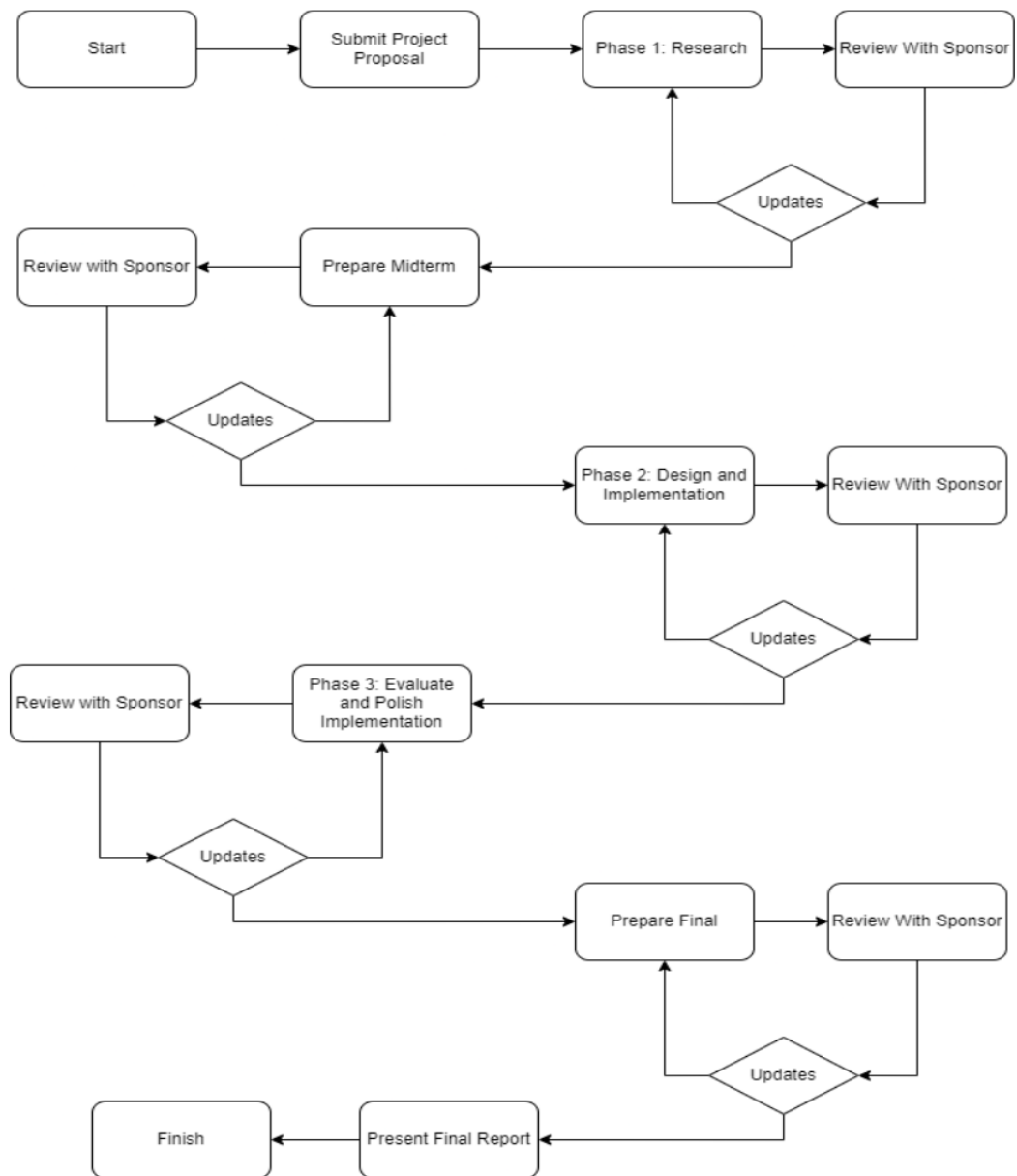


Figure 1: Research Plan Flowchart

7. Anticipated Issues and Challenges

This project is poised to encounter a series of challenges, necessitating proactive planning and resolution strategies. Firstly, the smaller team size could potentially lead to a heavier workload for individual members, demanding efficient resource allocation and time management. Additionally, the group's collective expertise in malware analysis and reverse engineering is varied, potentially resulting in differing levels of proficiency. This diversity of skills may require additional effort in knowledge sharing and collaborative learning to ensure that all team members are adequately equipped for the tasks at hand.

Furthermore, there may be a notable gap in domain-specific knowledge among team members, potentially impacting the depth of analysis required to extract meaningful insights from the available data. Addressing this challenge will involve targeted training and resource acquisition to bolster the team's collective knowledge base. Moreover, the compatibility of virtual environments across different operating systems, especially Windows and M1 Mac Machines, may pose logistical challenges. Ensuring seamless functionality and support for both Windows and M1 Mac environments will be crucial in maintaining an effective workflow throughout the project.

Finally, there is the potential for our initially proposed methods to face limitations in extracting artifacts from the Malware Development Life Cycle (MDLC) and establishing malware similarity. In the event that these methods do not yield the expected results, we will rely on our research and adapt our approach to a more effective alternative. These anticipated challenges underscore the importance of a dynamic and adaptable project strategy, with a focus on collaborative learning and resource optimization.

8. Broader Impact

Malware analysis is the analysis of a suspicious file by using tools and procedures in order to understand its behavior and purpose. The team is focusing on similarity analysis between different malware, which could be helpful for security teams rapidly detect potential malicious objects before any destruction happens within the organization. By understanding similarities between malware, ultimately, the team's research will exert impact on improving the accuracy of categorizing and evaluating malware attack incidents.

Moreover, malware similarity analysis also benefits malware analysis automation. Over the years, malware evolves not only in complexity but also diversity and quantity. A better understanding of malware similarities can improve the speed and accuracy of malware detection automation.

9. Final Progress

The core of our project lies in the meticulous examination of suspicious files, employing a diverse set of tools and methodologies to unravel their behavior and objectives. Our team is particularly dedicated to conducting similarity analyses among distinct malware instances, a crucial initiative with the potential to proactively identify potential malicious entities before they can pose any threat to the organization. By gaining insights into the shared characteristics of different malware strains, our research aims to significantly improve the precision of categorizing and assessing malware attack incidents. Additionally, our focus on malware similarity analysis is geared towards automating aspects of the malware analysis process. In the evolving landscape of cybersecurity, where malware exhibits increasing complexity and diversity, a deeper understanding of similarities among malware specimens holds the key to enhancing the speed and accuracy of automated malware detection processes. Through our project, we aim to contribute not only to the theoretical foundations of malware analysis but also to the practical development of more efficient and effective cybersecurity measures.

Upon receiving the malware samples, our initial undertaking was to establish the foundational infrastructure for the project. This entailed comprehensive research on existing tools, and among the options available, we integrated PEstudio into our toolkit. PEstudio serves as a fundamental tool specifically designed for the examination of Portable Executable (PE) files. Its primary function revolves around providing valuable insights into the composition of PE files and identifying potential threats. As a crucial component of our analysis arsenal, PEstudio assists analysts in comprehending the intricate structure of PE files, uncovering possible indicators of compromise, and facilitating the swift identification of malicious elements within the examined samples.

9.1 Heuristics Phase 1

During this crucial stage of our design, we used a static analysis method to carefully review the lines handed by our sponsor. The main tool used for this was PEstudio, which allowed for a thorough analysis of movable Executable (PE) lines. To examine the PE lines in-depth, this required uploading them into PEstudio.

Structural Analysis

Thorough structural analysis of the PE lines was completed, covering critical elements such as heads, sections, and entry points. PEstudio made the structure of the train easier to see, giving judges a better idea of the executable's relationship.

Functionality Examination

Within the PE lines, a thorough examination of the imported and exported functions was done. The executable's external functions were connected, providing insight into the train's intended features.

Library and Dependency Identification

PEstudio was essential for letting s understand the dependencies between the libraries (DLLs) used by the PE lines.

Pointers of concession(IoC) Discovery

The tool was required to identify implicit points of concession (IoCs) by connecting questionable traits or patterns that might indicate aggressive gestures. This innovative method assisted judges in quickly identifying and evaluating potential pitfalls.

Security Risk Assessment

PEstudio carried out a comprehensive search for legal oddities or configurations that might point to security flaws in the PE lines. The goal of this critical analysis was to identify any patterns that might present security risks.

imports (155)	flag (29)	first-thunk-original (INT)	first-thunk (IAT)	hint	group (11)	technique (12)
WritePrivateProfileStringA	x	0x00007720	0x00007720	924 (0x039C)	registry	-
RegSetValueExA	x	0x00008034	0x00008034	505 (0x01F9)	registry	T1112 Modify Registry
RegEnumKeyA	x	0x00008012	0x00008012	469 (0x01D5)	registry	T1012 Query Registry
RegDeleteKeyA	x	0x00008028	0x00008028	464 (0x01D0)	registry	T1485 Data Destruction
RegDeleteValueA	x	0x00008066	0x00008066	466 (0x01D2)	registry	T1485 Data Destruction
RegCreateKeyExA	x	0x00008046	0x00008046	461 (0x01CD)	registry	T1112 Modify Registry
SearchPathA	x	0x00007894	0x00007894	720 (0x02D0)	reconnaissance	-
MoveFileA	x	0x000078CA	0x000078CA	612 (0x0264)	file	T1105 Remote File Copy
SetFileAttributesA	x	0x00007928	0x00007928	782 (0x030E)	file	-
RemoveDirectoryA	x	0x00007A70	0x00007A70	690 (0x02BA)	file	-
WriteFile	x	0x00007748	0x00007748	919 (0x0397)	file	-
FindNextFileA	x	0x0000770E	0x0000770E	211 (0x00D3)	file	T1083 File and Directory Discovery
FindFirstFileA	x	0x000076FC	0x000076FC	201 (0x00CB)	file	T1083 File and Directory Discovery
DeleteFileA	x	0x000076EE	0x000076EE	124 (0x007C)	file	T1485 Data Destruction
SHGetPathFromIDListA	x	0x00007FC0	0x00007FC0	180 (0x00BC)	file	-
SHBrowseForFolderA	x	0x00007FAA	0x00007FAA	121 (0x0079)	file	-
SHGetFileInfoA	x	0x00007F98	0x00007F98	172 (0x00AC)	file	-
SHFileOperationA	x	0x00007F74	0x00007F74	154 (0x009A)	file	-
SHGetSpecialFolderLocation	x	0x00007FD8	0x00007FD8	190 (0x00C2)	file	-
GetCurrentProcess	x	0x00007988	0x00007988	314 (0x013A)	execution	T1057 Process Discovery
CreateProcessA	x	0x00007A5E	0x00007A5E	96 (0x0060)	execution	T1106 Execution through API
GetExitCodeProcess	x	0x000077EA	0x000077EA	338 (0x0152)	execution	-
ShellExecuteA	x	0x00007E88	0x00007E88	263 (0x0107)	execution	T1106 Execution through API
CloseClipboard	x	0x00007D60	0x00007D60	66 (0x0042)	data-exchange	T1115 Clipboard Data
SetClipboardData	x	0x00007D62	0x00007D62	585 (0x0249)	data-exchange	T1115 Clipboard Data
EmptyClipboard	x	0x00007E06	0x00007E06	193 (0x00C1)	data-exchange	T1115 Clipboard Data
OpenClipboard	x	0x00007E18	0x00007E18	501 (0x01F5)	data-exchange	T1115 Clipboard Data
SetCurrentDirectoryA	x	0x000078D6	0x000078D6	767 (0x02FF)	-	-

www.winit0r.com - [c:\users\acer\onedrive\desktop\specialtopics\winit0r\20161124\webrtc\setup.exe] - [read-only]

IP	indicator (21)	detail	level
	file > signature	Nullsoft PIMP Stub -> SFX	1
	file > embedded	signature: Nullsoft, location: overlay, offset: 0x0000B600, size: 54846 (byt...	1
	virustotal > score	2/71	1
	imports > flag > count	29	1
	overlay > file-ratio	54.07%	2
	overlay > size	54846 bytes	2
	overlay > entropy	7.997	2
	file > checksum	0x00000000	2
	sections > virtualized	.ndata	2
	groups > API	file reconnaissance diagnostic execution dynamic-library memory...	2
	string > URL	http://nsis.sf.net/NSIS_Error	2
	mitre > technique	T1105 T1497 T1124 T1057 T1106 T1083 T1082 T1485 T1055 T111...	2
	file > entropy	7.487	3
	file > sha256	BE60CC98261AB3C44D91E5EE0B4E9F24894DB6F9987D1A28F70C4411261...	3
	file > size	101438 bytes	3
	rich-header > checksum	0xFB2414A1	3
	rich-header > offset	0x00000000	3
	rich-header > footprint	F4079DB2511051EF8BE4946CD0AC9E319050929C216836EBF412E10D1D0...	3
	file > tooling	Visual Studio 2003	3
	file > subsystem	GUI	3
	imports > ordinal > count	1	3

9.2 Heuristics Phase 2

Our method addresses the inherent issues with traditional hashing techniques by using piecewise hashing to provide more detailed file analysis, lower the chance of collisions, and improve the accuracy of malware identification. The algorithm uses context triggering to adjust its hashing strategy according to the properties of the file, guaranteeing a dynamic method of capturing complex relationships. Advanced techniques for calculating similarity allow for a more deeper comprehension by identifying files that are almost identical from those that are only somewhat similar. A key component of our approach is fuzzy hashing, which adds adaptability to hash comparisons to adjust for small file differences. The limitations of generic hashing techniques are addressed by this extensive hashing technique, which offers cybersecurity experts a reliable way to more accurately identify and analyze similar but not identical files.

9.3 Heuristics Phase 3

The final phase was discovering about fuzzy Hashing and how can we do that for the malware samples. We used the fuzzy hashing technique to see if and find similar or identical content in files. Fuzzy hashing, in particular through ssdeep's context-triggered piecewise hashing, produces hashes that capture the file's structure and content in contrast to traditional hashing techniques that produce an exact hash match for binary files. The technique used by ssdeep to generate these fuzzy hashes makes it possible to identify content even when two files are similar but not exactly the same. We have found that this sophisticated hashing method is very helpful in identifying trends and commonalities across a variety of datasets. Notable for its consistency in identifying similarity and offering a useful solution for content comparison, ssdeep was a vital tool in our fuzzy hashing efforts.

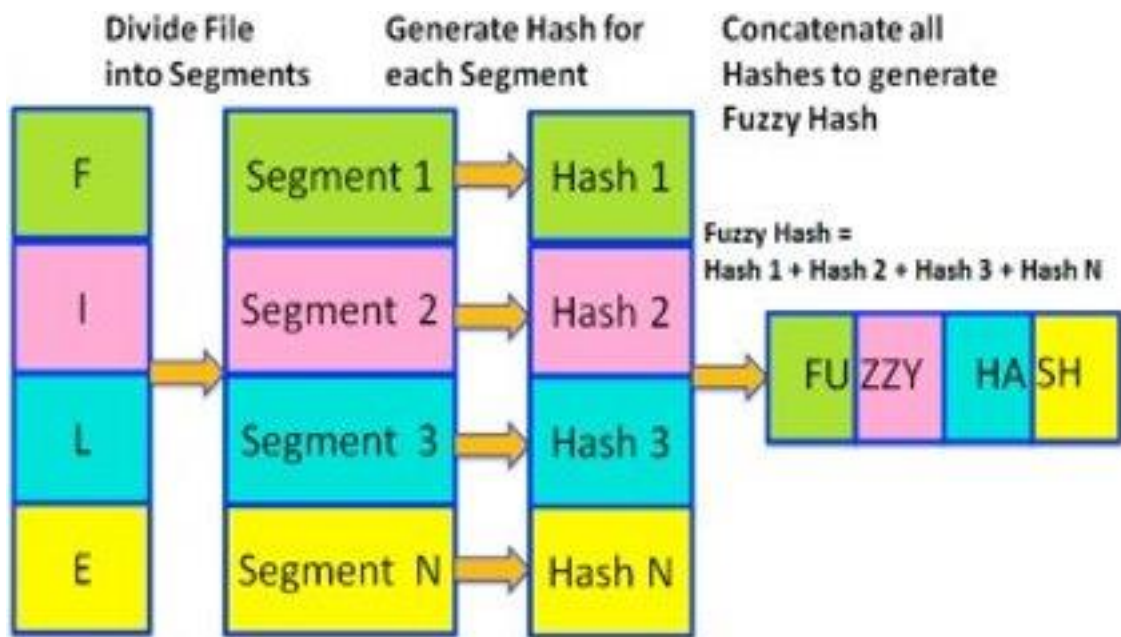


Figure 9.3: Fuzzy Hashing

9.4 SSDeep

When it comes to classifying and linking related malware samples, ssdeep is a useful tool in the field of malware analysis. Through enabling the grouping of malware variants that display comparable traits, ssdeep plays a crucial role in illuminating the connections and relationships among various malware strains. We worked with ssdeep and installed the tool and completed the setup process. Frank kindly shared a directory containing malware samples, which we ran ssdeep on in order to produce fuzzy hashes. and ascertained the files' similarity percentage. Combined with the ssdeep malware analysis tool, this proactive approach shows our dedication to accurate and thorough documentation.

9.5 functionality of Ssdeep tool

We used the installed ssdeep tool to analyze six spam emails that were chosen for our input data section based on their inherent similarities. We then calculated the hash code for every email and methodically saved all of the hash codes that were produced for these six files in a specific file called fuzzyHashingSimilarity.ssd.

```
@Venus-Air FuzzyHashing % ssdeep -h
ssdeep version 2.14.1 by Jesse Kornblum and the ssdeep Project.
copyright information, see man page or README.TXT.

Usage: ssdeep [-m file] [-k file] [-dpgvrsblcxa] [-t val] [-h|-V] [FILES]
  Match FILES against known hashes in file
  Match signatures in FILES against signatures in file
  Directory mode, compare all files in a directory
  Pretty matching mode. Similar to -d but includes all matches
  Cluster matches together
  Verbose mode. Displays filename as its being processed
  Recursive mode
  Silent mode; all errors are suppressed
  Uses only the bare name of files; all path information omitted
  Uses relative paths for filenames
  Prints output in CSV format
  Compare FILES as signature files
  Display all matches, regardless of score
  Only displays matches above the given threshold
  Display this help message
  Display version number and exit
@Venus-Air FuzzyHashing % ls
Email1.pages      spamEmail2.pages      spamEmail3.pages      spamEmail4.pages      spamEmail5.pages
@Venus-Air FuzzyHashing % ssdeep * > fuzzyHashingSimilarity.ssd
@Venus-Air FuzzyHashing % cat fuzzyHashingSimilarity.ssd
ssdeep,1.1--blocksize:hash:hash,filename
:6dsurj2MD+oIfk3jHPWjTTrT+jzZ4qUf6WL7Aj:Lun2ujIfk3zgT+XzyVLkj,"/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages
:Hsurj2MDvoLs/ujK3GGQXt8PuaLW21pkL:Mun2uwlS/AK3GGQTrTU,"/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages
:kj0surj2MDvogn9LoD9NfPOwdYbQdf8nX78fKJn1w:Kxun2uwn9Li9NnO+4Kf8nX78fQw,"/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages
:dU1LsqZ0Rk3sZ4bYVdTOx0R+lJ29niyhS4NHtTxk0n2ZxobqJKqF35u8zu+/3bxm:mhhHsurj2MDvouJl3u8hLAYBM,"/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages
:fHk3sZ4bYVdTOx0R+lJ29niyhS4NHtTxk0n2Zx07QAZcFhqtCzrdNiOjvTLDwAmP:+surj2MDvoMAm6CzBISP0Am5fBuOt,"/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages
:jSurj2MD+ohZLWPsnfPOwdYbQdf8nX78fKJnFm:Yun2ujhZLksNnO+4Kf8nX78f4m,"/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages
```

Figure 9.5: SSdeep

9.6 Findings

Utilized the ssdeep tool for Fuzzy Hashing to compare the files, uncovering the percentage of similarity among them. This approach proved effective in identifying similarities between emails, thereby enhancing the capability to detect digital forensics within the data.

```
venu@Venus-Air FuzzyHashing % ssdeep -m fuzzyHashingSimilarity.ssd * -s
/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages (100)
/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages (36)
/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages (38)
/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages (100)
/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages (36)
/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages (49)
/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages (40)
/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages (36)
/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages (100)
/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages (77)
/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages (36)
/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages (49)
/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages (100)
/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages (63)
/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages (36)
/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages (40)
/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages (63)
/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages (100)
/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail1.pages (38)
/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail2.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail3.pages (77)
/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail4.pages (36)
/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail5.pages (35)
/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages matches fuzzyHashingSimilarity.ssd:/Users/venu/Desktop/FuzzyHashing/spamEmail6.pages (100)
/Users/venu/Desktop/FuzzyHashing/-s: No such file or directory
```

Figure 9.6 Output of SSdeep

9.7 Summary

We created fuzzy hashes that captured the structure and content of files by utilizing ssdeep's context-triggered piecewise hashing technique. This made it possible to identify similarities even in non-identical files. Our efforts were greatly aided by the ssdeep tool, which is well-known for its reliability and potency in finding similarities across a variety of datasets. We generated fuzzy hashes using ssdeep by analyzing spam emails and malware samples. This allowed us to determine similarity percentages and improve our capacity to find digital forensics in the data. Using ssdeep proactively demonstrates our dedication to precise and comprehensive documentation in the field of malware investigation.

9.8 Challenges

This semester, our team faced quite a few of challenges that needed to be carefully navigated. First of all, working with a smaller team of just three people presented a special set of challenges that required closer cooperation and efficient use of available resources. The team also had to deal with issues arising from members' varying and limited domain-specific knowledge, which required a concentrated effort to close knowledge gaps and promote a more unified perspective.

Technical difficulties also arose, mostly related to attaining interoperability among diverse operating systems. It became more complex to handle Windows and MAC M1 computers in Linux and Windows virtual environments, which called for creative ways to ensure smooth integration. To be more precise, finding malware samples for fuzzy hashing was a major difficulty that required careful consideration in order to improve the procedure's effectiveness.

9.9 Future Work

All the challenges we discussed leave a lot of room for future work and many projects. The application of ssdeep Fuzzy hashing for similarity analysis showed certain difficulties in effectively analyzing similarities across distinctive images, despite clear visual similarities. Investigating the root causes of these discrepancies along with developing effective solutions stands out as an option for future research in this area of study.

Furthermore, exploring additional techniques known for their better time complexity might improve similarity detection accuracy. This suggests a more progressive approach for future research, focusing on the optimization of similarity analysis efficiency among various data sets. Addressing these issues could significantly help to enhance image similarity analysis methods.

10. Investigator Biographies

Venu Gopal Reddy: Second year of pursuing an MS in Computer Science at the University of Texas at Dallas, building upon my undergraduate degree in Electronics and Communication Engineering from NIT Manipur, India. With 2.5 years of professional experience as a Software Awareness Engineer in both India and the USA, my role as a Java Spring Backend Developer equipped me with proficiency in development tools such as Spring Tool Suite, DBeaver, and Postman. Currently interning at Copart as a Software Awareness Development Engineer, I am leveraging this opportunity to further expand my skill set. My expertise lies in Java, and I have also explored C/C++ and Python. This diverse background reflects my commitment to continuous learning and adaptability in the dynamic field of computer science, and my present internship at Copart is contributing significantly to this journey.

Neelima Reddy Kapa: A second-year student at UTD majoring in computer science, adding to my undergraduate degree from SRM University Chennai in computer science engineering. Practical experiences, such as an internship in data science at MIT during my undergraduate studies, have enhanced my academic path. This internship helped me grow a lot because it improved my Python skills and gave me practical experience with machine learning packages. I have gained a solid foundation in data structures and programming languages like C and C++ outside of my internship. Notably, I have actively participated in a number of machine learning projects, which has helped me improve my Python skills and acquire useful expertise managing machine learning models. My comprehension of the subject has grown as a result of these projects, which have also shown my capacity to translate abstract ideas into practical situations. This thorough mixture of academic and practical experiences underscores my dedication to the field of computer science and machine learning.

Navya Teja Gajula: As a computer science enthusiast, I earned my Bachelor's in Electronics and Communication from the National Institute of Technology in India, delving into diverse courses from Data Structures to Machine Learning. Pursuing my Master's at The University of Texas at Dallas, I engaged in hands-on projects, including predicting stock market trends with PySpark and recurrent neural networks. I explored natural language processing through text summarization using Latent Semantic Analysis and demonstrated my proficiency in web development with an E-commerce Website using React.js and Node.js, emphasizing scalability and modern technologies like Docker and Git. Beyond

academics, I actively contributed as a Programming Instructor at CS Outreach, UT Dallas, showcasing leadership and teamwork skills.

11. References

1. Eugene H Spafford, Stephen A Weeber, "SoAware Forensics: Can we track code to its authors?", Purdue Technical Report CSD-TR 92-010, 19 February 1992.
2. Frank Posluszny, "MTR 200010 - Malware Similarity Analysis: An adversary perspective", The MITRE Corporation, March 2020.
3. Intezer. (2021, December 22). The role of malware analysis in cybersecurity. Intezer. <https://www.intezer.com/blog/malware-analysis/the-role-of-malware-analysis-in-cybersecurity/>
4. Cassiusm, P., Annelie, H., Olivier, Z., and Lamine, N. (2021, August 17-20). Accurate and Robust Malware Analysis through Similarity of External Calls Dependency Graphs (ECDG). IWCC'21, All-Digital Conference